VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Shankar Shivappa Pujar (1BM23CS309)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2025 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Shankar Shivappa Pujar (1BM23CS309),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Ambuja k Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	30-01-2026	Write a python program to import and export data using Pandas library functions	3-6
2	06-02-2026	Demonstrate various data pre-processing techniques for a given dataset	7-15
3	13-02-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	16-22
4	27-02-2026	Build Logistic Regression Model for a given dataset	23-26
5	06-03-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	27-33
6	13-03-2026	Build KNN Classification model for a given dataset.	34-40
7	08-03-2026	Build Support vector machine model for a given dataset.	41-43
8	08-04-2026	Implement Random forest ensemble method on a given dataset.	44-47
9	08-04-2026	Implement Boosting ensemble method on a given dataset.	48-50
10	10-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	51-52
11	10-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	53-54

Github Link:
https://github.com/shankar045/1BM23CS309-ML-LAB

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

LAB 1

Demonstrate various data pre-processing techniques for given dataset

i) To load .csv file into the data frame.

```
df = pd.read_csv ("housing.csv")
```

ii) To display information of all coloumns

```
print (df.info())
```

iii) To display statistical information of all numerical

```
print (df.describe())
```

(IV) To display the count of unique label for ocean proximity coloumns

```
print (df['ocean proximity'].value_counts())
```

v) To display which attributes coloumns in a dataset have missing values count > 0

```
print (df.isnull().sum() [df.isnull().sum>
```

Summary

Data Analysis findings

The core task is to find missing value & categorical coloumns and how to handle them

Missing values: 'Glucose' 'Blood pressure', 'Skinthickness' 'Insulin'

Categorical coloumns: None (outcome was 'Numerical' (0s replaced with Nan then median imputed)

2. Adult Income Dataset

Missing values: 'workclass', 'occupation', 'native-country' replaced with NaN, then mode imputed

categorical coloumns: 'Income' (label_encoded): other (eg: 'workclass', 'education') one-hot_encoded

Scaling differences

Min - max Scaling: Scales to 0 to 1 range
Good for range sensitive algorithm sensitive to outliers

Standarization: 'Scales to mentioned 1 Good normally distributed data better with outliers

Code:

```
import pandas as pd

import os


path = "your_folder_path_here"

df = pd.read_csv(os.path.join(path, "housing.csv"))


print(df.info())

print(df.isnull().sum())

df["total_bedrooms"].fillna(df["total_bedrooms"].median(), inplace=True)

print(df.isnull().sum())

df = pd.get_dummies(df, columns=["ocean_proximity"])

X = df.drop("median_house_value", axis=1)

y = df["median_house_value"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

print(X_train.shape)

print(X_test.shape)
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot:

LAB-2

Demonstrate various data pre-processing techniques for a given dataset

1. load dataset

```
url = "https://raw.githubusercontent.com/ageron/handson-ml
        master/datasets/housing/housing.csv"
data = pd.read_csv(url)
```

2. Basic info

```
print(data.head(10))
print(data.dtypes)
print(data.describe().T)
print(data.median())
```

~~plt.fi~~

3. Target distribution

```
plt.figure(figsize=(8,4))
plt.hist(data['median_house_value'], bins=50, color="sk
plt.title("Histogram of median house value")
plt.show()
sns.boxplot(x=data['median_house_value'], color="light
plt.show()
```

. missing values

~~print("total bedrooms~~

```
print(data.isnull().sum())
print(data.isnull().sum()/len(data)*100)
```

```
data["total_bedrooms"].fillna(data["total_bedrooms"].
    median(), inplace=True)
```

6. One hot encode ocean proximity

```
data = pd.get_dummies(data, columns=["ocean_proximity"])
```

7. Feature Scaling

```
num_cols = data.select_dtypes(include=["int64","float64"]).
            columns.drop("median_house_value")
Scalar = StandardScalar()
data[num_cols] = scalar.fit_transform(data[num_cols])
```

8. Random train-test split

```
X = data.drop("median_house_value", axis=1)
y = data["median_house_value"]
X_train, X_test, y_train, y_test = train_test_split(X,
    test_size=0.2, random_state=42)
```

9. linear Regression

```
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
y_pred = lin_reg.predict(X_test)
```

10. Evaluate

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"RMSE: {rmse:.2f}, MAE: {mae:.2f} R²: {r2:.2f}")
```

11. Correlation

```
corr = data.corr()
plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

12. Scatter plot median_income vs median_house_value

```
plt.figure(figsize=(8,6))
plt.scatter(data["median_income"], data["median
    house_value"], alpha=0.5)
plt.xlabel("median income")
plt.ylabel("Median house value")
plt.show()
```

Output

columns & types

float64 : 8

object : 1

dtype : int64

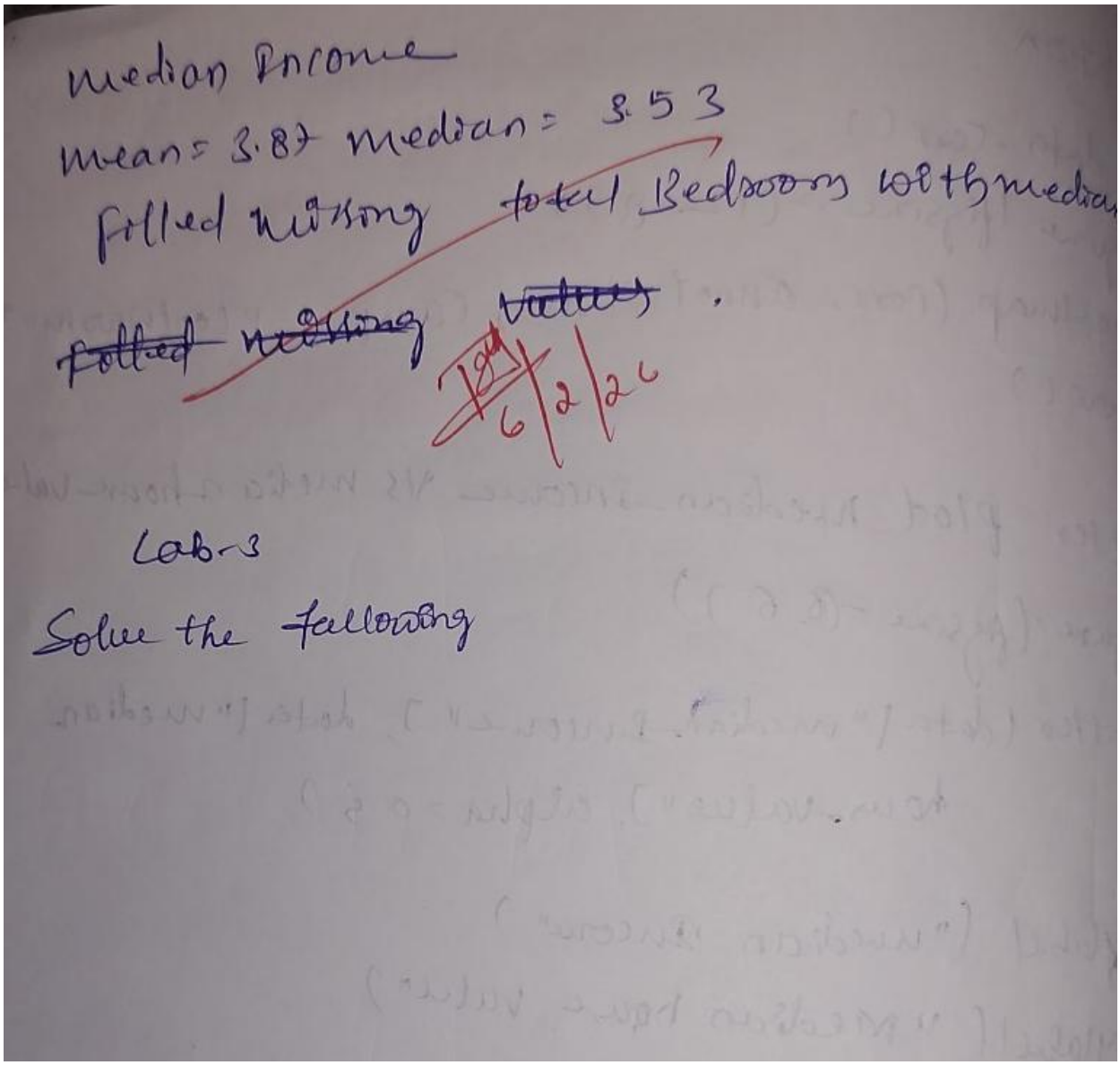
missing values

total bedrooms : 207

Target variable : 'median_house_value'

Mean = 206853, median = 179700

min = 14999, max = 500001



median Income

mean = 3.87 median = 3.53

filled missing total Bedrooms with median

~~filled missing values~~

6/2/26

Lab-3

Solve the following

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedShuffleSplit

from sklearn.preprocessing import MinMaxScaler, StandardScaler, OneHotEncoder
```

```
from sklearn.impute import SimpleImputer

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"

housing = pd.read_csv(url)

print("First 10 rows of the dataset:")

print(housing.head(10))

print("\nData types of all attributes:")

print(housing.dtypes)

basic_stats = housing.describe().T

print(basic_stats)

plt.figure(figsize=(8, 5))

plt.hist(housing["median_house_value"], bins=50, color="skyblue", edgecolor="black")

plt.xlabel("Median House Value")

plt.ylabel("Frequency")

plt.title("Histogram of Median House Value")

plt.show()


plt.figure(figsize=(8, 3))

sns.boxplot(x=housing["median_house_value"], color="lightgreen")

plt.xlabel("Median House Value")

plt.title("Box Plot of Median House Value")

plt.show()
```

```
missing_counts = housing.isnull().sum()
missing_percentage = (missing_counts / len(housing)) * 100
missing_summary = pd.DataFrame({
    "Missing Values": missing_counts,
    "Percentage (%)": missing_percentage
})
print(missing_summary)
median_bedrooms = housing["total_bedrooms"].median()
housing["total_bedrooms"].fillna(median_bedrooms, inplace=True)
print(housing["total_bedrooms"].isnull().sum())
housing_encoded = pd.get_dummies(housing, columns=["ocean_proximity"])
print(housing_encoded.columns)
minmax_scaler = MinMaxScaler(feature_range=(0, 1))
num_cols = housing_encoded.select_dtypes(include=["int64", "float64"]).columns
housing_encoded[num_cols] = minmax_scaler.fit_transform(housing_encoded[num_cols])
housing_encoded.head()
housing["income_cat"] = pd.cut(
    housing["median_income"],
    bins=[0.0, 1.5, 3.0, 4.5, 6.0, np.inf],
    labels=[1, 2, 3, 4, 5]
)
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)


for train_idx, test_idx in split.split(housing, housing["income_cat"]):
```

```
    strat_train_set = housing.loc[train_idx].drop("income_cat", axis=1)
    strat_test_set = housing.loc[test_idx].drop("income_cat", axis=1)

print("Training set size:", len(strat_train_set))
print("Test set size:", len(strat_test_set))

num_attributes = strat_train_set.select_dtypes(include=["int64", "float64"])
corr_matrix = num_attributes.corr()
print(corr_matrix["median_house_value"].sort_values(ascending=False))

plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Correlation Matrix Heatmap")
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(
    strat_train_set["median_income"],
    strat_train_set["median_house_value"],
    alpha=0.5,
    color="teal"
)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Scatter Plot: Median Income vs Median House Value")
```

```
plt.show()

X = housing_encoded.drop("median_house_value", axis=1)

y = housing_encoded["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


print("Training set size:", X_train.shape)

print("Test set size:", X_test.shape)


corr_target = X_train.join(y_train).corr()["median_house_value"].sort_values(ascending=False)


top3_positive = corr_target[1:4]

print("Top 3 positively correlated features:\n", top3_positive)

most_negative = corr_target[-1:]

print("\nMost negatively correlated feature:\n", most_negative)

scaler = StandardScaler()

X_train_scaled = X_train.copy()

X_test_scaled = X_test.copy()

num_cols = X_train.select_dtypes(include=["int64", "float64"]).columns

X_train_scaled[num_cols] = scaler.fit_transform(X_train[num_cols])

X_test_scaled[num_cols] = scaler.transform(X_test[num_cols])

X_train_scaled.head()

num_cols = X_train.select_dtypes(include=["int64", "float64"]).columns

cat_cols = X_train.select_dtypes(include=["object"]).columns
```

```
preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), num_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), cat_cols)
])
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)print(f"RMSE: {rmse:.2f}, MAE: {mae:.2f}, R^2: {r2:.4f}")
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot:

Lab - 3

Solve the following linear regression problem using Matrix Approach

Find the linear regression of the data of week and product sales

x: week	y: (sales in thousand)
1	2
2	4
3	5
4	9

$$\beta = ((x^T x)^{-1} x^T) y$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} + \begin{bmatrix} E_1 \\ E_2 \\ \vdots \\ E_n \end{bmatrix}$$

$$Y = X\beta \cdot \quad X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

$$x^T x = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$(x^T x)^{-1} = \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{20}\begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}$$

$$(X^T X)^{-1} X^T = \frac{1}{20}\begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

$$= \frac{1}{20}\begin{bmatrix} 20 & 10 & 0 & -10 \\ -6 & -2 & 2 & 6 \end{bmatrix}$$

$$((X^T X)^{-1} X^T) y = \frac{1}{20}\begin{bmatrix} 20 & 10 & 0 & -10 \\ -6 & -2 & 2 & 6 \end{bmatrix}\begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

$$= \frac{1}{20}\begin{bmatrix} 40+40-90 \\ 12-8+10+54 \end{bmatrix}$$

$$= \frac{1}{20}\begin{bmatrix} -10 \\ 44 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix}$$

$$\beta = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix} \quad \beta_0 = -0.5 \quad \beta_1 = 2.2 \qquad y = -0.5 + 2.2x$$

Output

1. did you perform any steps

→ yes

2, Canada per - capital. csv. did you visualize. The regression line along with the data points what does the plot tell you about the relationship between year and per. Capital income

→ yes the regression line was plotted. along with the data points

The plot shows a positive linear Relationship b/w year & per - income. As the year increase The per capital income also increase steadily

This candidates strong upward linear Trend

3, For Hiring.csv what the predicted. for a candidate with 12 years of Experience 10 test score and 10 Interview score

→ [53290.89255945]

[2268.07229784]

4) For 1000- Companies.csv did you Encode Categorical variables (Eg state)? If yes ~~How did you the features~~? If yes why

→ yes categorical Variable state was Encoding Using one-hot Encoding (pd.get_dummies) this converts state names into Binary columns (0 and 1) so the model can process them Numerically

feature Scaling was not performed Because linear regression does not require feature Scaling to function Correctly

1→ yes, preprocessing was performed

1) Canada-per-capita-income.csv → removed missing values using dropna() to ensure clean Numerical input

2, Salary.csv → removed missing values using dropna() to avoid ~~Errors~~ during model ~~Errong~~

→ hiring.csv → yes

Day 6/3/26

Code:

```
import pandas as pd

import numpy as np

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import LabelEncoder

import matplotlib.pyplot as plt

df = pd.read_csv('housing_area_price.csv')

plt.xlabel('area')

plt.ylabel('price')

plt.scatter(df.area, df.price, color='red', marker='+')

new_df = df.drop('price', axis='columns')

price = df.price

reg = LinearRegression()

reg.fit(new_df, price)

reg.predict([[3300]])

reg.coef_

reg.intercept_

3300 * reg.coef_[0] + reg.intercept_

reg.predict([[5000]])


df = pd.read_csv('homeprices_Multiple_LR.csv')

df.bedrooms = df.bedrooms.fillna(df.bedrooms.median())

reg = LinearRegression()

reg.fit(df.drop('price', axis='columns'), df.price)
```

```
reg.coef_

reg.intercept_

reg.predict([[3000, 3, 40]])

112.06244194*3000 + 23388.88007794*3 + -3231.71790863*40 + 221323.00186540384

df = pd.read_csv('canada_per_capita_income.csv')

print(df.head())

X = df[['year']]

y = df['per capita income (US$)']

reg = LinearRegression()

reg.fit(X, y)

predicted_income_2020 = reg.predict([[2020]])

print(predicted_income_2020[0])


plt.scatter(X, y)

plt.plot(X, reg.predict(X))

plt.xlabel('Year')

plt.ylabel('Per Capita Income')

plt.title('Per Capita Income in Canada Over the Years')

plt.show()


df = pd.read_csv('salary.csv')

print(df.head())

print(df.isnull().sum())
```

```
df['YearsExperience'] = df['YearsExperience'].fillna(df['YearsExperience'].median())
print(df.isnull().sum())

X = df[['YearsExperience']]
y = df['Salary']

reg = LinearRegression()
reg.fit(X, y)

predicted_salary_12_years = reg.predict([[12]])
print(predicted_salary_12_years[0])

plt.scatter(X, y)
plt.plot(X, reg.predict(X))
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Salary vs Years of Experience')
plt.show()

def convert_to_numeric(value):
    word_to_num = {
        'zero': 0, 'one': 1, 'two': 2, 'three': 3, 'four': 4, 'five': 5,
        'six': 6, 'seven': 7, 'eight': 8, 'nine': 9, 'ten': 10,
        'eleven': 11, 'twelve': 12, 'thirteen': 13, 'fourteen': 14,
```

```
        'fifteen': 15
    }
    return word_to_num.get(value.lower(), value) if isinstance(value, str) else value


df_hiring = pd.read_csv('hiring.csv')
print(df_hiring.head())


df_hiring['experience'] = df_hiring['experience'].apply(convert_to_numeric)
df_hiring['experience'].fillna(0, inplace=True)
df_hiring['test_score(out of 10)'].fillna(df_hiring['test_score(out of 10)'].median(), inplace=True)
df_hiring['interview_score(out of 10)'].fillna(df_hiring['interview_score(out of 10)'].median(),
inplace=True)


X_hiring = df_hiring[['experience', 'test_score(out of 10)', 'interview_score(out of 10)']]
y_hiring = df_hiring['salary($)']
reg_hiring = LinearRegression()
reg_hiring.fit(X_hiring, y_hiring)
candidates = np.array([[2, 9, 6], [12, 10, 10]])
predicted_salaries = reg_hiring.predict(candidates)
for i, candidate in enumerate(candidates):
    print(predicted_salaries[i])
plt.scatter(y_hiring, reg_hiring.predict(X_hiring))
plt.xlabel("Actual Salary")
plt.ylabel("Predicted Salary")
plt.title("Actual vs Predicted Salary")
```

```
plt.show()

df_companies = pd.read_csv('1000_Companies.csv')

print(df_companies.head())

label_encoder = LabelEncoder()

df_companies['State'] = label_encoder.fit_transform(df_companies['State'])


df_companies.fillna(df_companies.median(), inplace=True)

X_companies = df_companies[['R&D Spend', 'Administration', 'Marketing Spend', 'State']]

y_companies = df_companies['Profit']

reg_companies = LinearRegression()

reg_companies.fit(X_companies, y_companies)

input_data = np.array([[91694.48, 515841.3, 11931.24, label_encoder.transform(['Florida'])[0]]])

predicted_profit = reg_companies.predict(input_data)

print(predicted_profit[0])

plt.scatter(y_companies, reg_companies.predict(X_companies))

plt.xlabel("Actual Profit")

plt.ylabel("Predicted Profit")

plt.title("Actual vs Predicted Profit")

plt.show()
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot:

Lab = 4

Logistic Regression

1. logistic Regression problem.

$a_0 = -5$

$a_1 = 0.8$

Study hours = x.

(a) logistic Regression · Equation

linear Eqn $z = a_0 + a_1 x$.

$z = -5 + 0.8x$

$P(\text{pass}) = \frac{1}{1+e^{-z}}$

$P(\text{pass}) = \frac{1}{1+e^{(-5+0.8x)}}$

$z = -5 + 0.8(7)$

$z = -5 + 5.6$

$z = 0.6$

$P = \frac{1}{1+e^{0.6}}$

$e^{0.6} = 0.5488$

$P = \frac{1}{1.5488}$

$P = 0.645$

$P(\text{pass}) \approx 0.645$

$p = \frac{\quad}{1.5488}$

$p \approx 0.645$

$P(\text{pass}) \approx 0.645$

So the

(c) predict class (threshold = 0.5)

If $p \geq 0.5 \rightarrow$ pass

If $p \leq 0.5 \rightarrow$ fail

$0.645 > 0.5$ = pass.

2. Softmax problem

$z = [2, 1, 0]$

$p_i = \frac{e^{z_i}}{\sum e^{z_j}}$

Step 1 Compute Exponential

$e^2 \approx 7.389$

$e^1 \approx 2.718$

$e^0 = 1$

Step 2 Compute Sum

$7.389 + 2.718 + 1 = 11.107$

Step 3 Compute probabilities

Class 1 $p_1 = \frac{7.389}{11.107} \approx 0.665$

Class 2 $p_2 = \frac{2.718}{11.107} \approx 0.245$

Class 3 $p_3 = \frac{1}{11.107} \approx 0.090$

Final Softmax probabilities

[0.665, 0.245, 0.090]

Highest probability = Class 1

HR → Dataset Binary logistic Regression:

```
import pandas as pd.
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv('HR comma_sep.csv')
df = pd.get_dummies(df, columns = ['Salary', 'Department'])
X = df[['satisfaction_level', 'average_montly_hours',
        'promotion last years', 'Salary_low', 'Salary_medium']]
```

Dataset

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix


df = pd.read_csv("HR_comma_sep.csv")


print(df.head())

print(df.corr(numeric_only=True))


pd.crosstab(df.salary, df.left).plot(kind='bar')

plt.title("Salary vs Retention")

plt.show()


pd.crosstab(df.Department, df.left).plot(kind='bar', figsize=(10,5))

plt.title("Department vs Retention")

plt.show()


df = pd.get_dummies(df, columns=['salary', 'Department'], drop_first=True)


X = df[['satisfaction_level',
```

```
        'average_montly_hours',
        'promotion_last_5years',
        'salary_low',
        'salary_medium']]

y = df['left']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:

3. Information gain for a3

- High: 4 Samples (0 yes 4 NO) → Entropy = 0
- Normal (1 yes, 0 NO) → Entropy = 0

$E(a_3) = 0$

$Sg(a_3) = 0.92$

∴ Best splitting attribute : a3

Implimentation

for iris dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn tree import Decision Classifier
from sklearn.metrics import accuracy_score, confusion matrix
from sklearn preprocessing import label Encoder

iris = pd.read_csv("/content/iris.csv.csv")
X = iris.iloc[:, :-1]
y = iris.iloc[:, -1]
Xtr, Xte, ytr, yte = train_test_split(X, y test_size=0.3,
    random test =0)
Model = Decision tree Classifier()
Model.fit (Xtr, Xte)
Model = model.predict(Xte)
Print ('Iris Accuracy:", accuracy_score(yte, pred))
print ('IRIS confusion matrix:\n", confusion_matrix(yte, pred))
```

Drug data set

```
drug = pd.read_csv ("/content/drug(1).csv")
le = label.Encoder()
for c in drug.columns:
    if drug[c].dtype == "object": drug[c] = le.fit_transform(drug[c])
X = drug.drop ("Drug", axis = 1)
Y = drug ["Drug"]
Xtr, Xte, ytr, yte = train_test_split (X, y, test_size=0.
                    random_state =0 )
Model = Decision tree Classifier()
model.fit (Xtr, ytr)
pred = model.predict (Xte)
print ("\n")
print ("Drug Accuracy:", accuracy_score (yte, pred))
print ("Drug confusion Matrix:\n", confusion_matrix (yte, pred))
```

OUTPUT

Iris Accuracy = 1.0

IRIS Confusion Matrix:

```
[[11  0  0]
 [0  13  0]
 [0  0  6]]
```

Drug Accuracy : 1.0

Drug confusion Matrix:

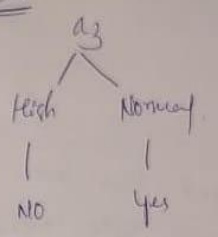
```
[[3  0  0  0 0]
 [0  1  0  0 0]
 [0  0  3  0]
 [0  0  0  4 0]
 [0  0  0  19]]
```

```
data = pd.read_csv("/content /petrol_consumption.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_
                                   size=0.2, random_state=
Model = DecisionTreeClassifier()
model.fit(X_train, y_train)

prediction
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)
print("Root mean Squared Error:", rmse)
```

Output

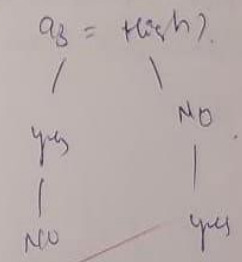
```
Mean Absolute Error: 50.6
Mean Squared Error: 4573.0
Root Mean Squared Error: 67.623
```

ID3



- if a_3 = High → classification = NO
- if a_3 = Normal → classification = yes

CART



- if a_3 = High → class = NO
- if $a_3 \neq$ High (Normal) → class = yes

13/3/26

Code:

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score
from sklearn.preprocessing import LabelEncoder, StandardScaler

def train_and_evaluate_iris():
    iris_df = pd.read_csv("iris.csv")
    X = iris_df.drop(columns=["species"])
    y = iris_df["species"]

    y_le = LabelEncoder()
    y = y_le.fit_transform(y)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
```

```
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)

    print("IRIS Dataset Classification:")
    print(f"Accuracy Score: {acc:.4f}")
    print(f"Precision Score: {prec:.4f}")
    print(f"Recall Score: {rec:.4f}")
    print(f"F1 Score: {f1:.4f}")

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=y_le.classes_,
                yticklabels=y_le.classes_)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix: iris.csv")
    plt.show()

train_and_evaluate_iris()
def train_and_evaluate_drug():
    drug_df = pd.read_csv("drug.csv")

    categorical_features = ["Sex", "BP", "Cholesterol"]
    label_encoders = {}
```

```
for col in categorical_features:
    le = LabelEncoder()
    drug_df[col] = le.fit_transform(drug_df[col])
    label_encoders[col] = le

X = drug_df.drop(columns=["Drug"])
y = drug_df["Drug"]

y_le = LabelEncoder()
y = y_le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted')
rec = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
cm = confusion_matrix(y_test, y_pred)
```

```
    print("Drug Dataset Classification:")
    print(f"Accuracy Score: {acc:.4f}")
    print(f"Precision Score: {prec:.4f}")
    print(f"Recall Score: {rec:.4f}")
    print(f"F1 Score: {f1:.4f}")
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=y_le.classes_,
                yticklabels=y_le.classes_)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix: drug.csv")
    plt.show()

train_and_evaluate_drug()
petrol_df = pd.read_csv("petrol_consumption.csv")
X = petrol_df.drop(columns=["Petrol_Consumption"])
y = petrol_df["Petrol_Consumption"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
```

```
)

model = DecisionTreeRegressor(max_depth=5, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Petrol Consumption Regression:")
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Program 6

Build KNN Classification model for a given dataset.

Lab 5

KNN classification Model for given dataset

for iris data-set

```
1) import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import ~~accuracy~~ KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

load dataset
data = pd.read_csv("iris (1).csv")
X = data.drop("Species", axis=1)
y = data['Species']

# train test split
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)

#KNN model
Knn = KNeighborsClassification(n_neighbors=5)
Knn.fit(X_train, y_train)

# prediction
y_pred = Knn.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
# classification report
print("Classification Report")
print(classification_report(y_test, y_pred))
```

For diabetes dataset

```
2. data = pd.read_csv('diabetes.csv')
X = data.drop("outcome", axis=1)
y = data['outcome']
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)

# feature Scaling
Scaler = StandardScaler()
X_train = Scaler.fit_transform(X_train)
X_test = Scaler.transform(X_test)

# KNN model
Knn = KNeighborsClassification(n_neighbors=7)
Knn.fit(X_train, y_train)
y_pred = Knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

1) How to choose the k-value Demonstrate using accuracy and Error rate

→ k value choosen by taking different values of k and sensitive minimum Error rate and give maximum accuracy.

accuracy rate = $\frac{\text{Correct prediction}}{\text{total prediction}}$

Error rate = 1 − Accuracy

) What is the purpose of feature scaling

feature Scaling is used to normalize the so that the All feature have the same scale in KNN distance is calculated b/w the data so large value can dominate Scalar value feature Scaling prevent the problem.

Ex:

Age = 90

Glucose = 150

without Scaling Glucose value dominate the distance calculation

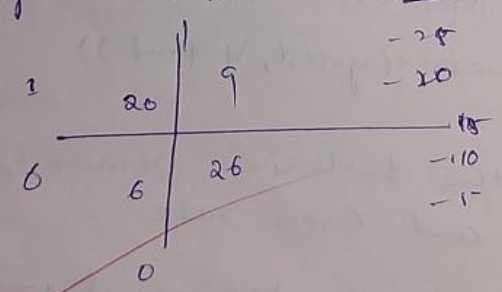
feature Scaling can be done using StandScale

Formula : $Z = \frac{x - \mu}{\sigma}$

output

Best K = 11

Accuracy : 0.75409836



✓ 6/3/26

	Precision	recall	f1-Score	Support
0	0.77	0.69	0.73	29
1	0.74	0.81	0.78	32
Accuracy			0.75	61
Macro avg	0.76	0.75	0.75	61
Weighted Avg	0.76	0.75	0.75	61

Code:

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

from sklearn.preprocessing import LabelEncoder, StandardScaler

def train_and_evaluate_iris():

    iris_df = pd.read_csv("iris.csv")

    X = iris_df.drop(columns=["species"])

    y = iris_df["species"]


    y_le = LabelEncoder()

    y = y_le.fit_transform(y)


    X_train, X_test, y_train, y_test = train_test_split(

        X, y, test_size=0.2, random_state=42

    )

    model = DecisionTreeClassifier(random_state=42)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
```

```
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)
    print("IRIS Dataset Classification:")
    print(f"Accuracy Score: {acc:.4f}")
    print(f"Precision Score: {prec:.4f}")
    print(f"Recall Score: {rec:.4f}")
    print(f"F1 Score: {f1:.4f}")
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=y_le.classes_,
                yticklabels=y_le.classes_)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix: iris.csv")
    plt.show()
train_and_evaluate_iris()


def train_and_evaluate_drug():
    drug_df = pd.read_csv("drug.csv")

    categorical_features = ["Sex", "BP", "Cholesterol"]
```

```
label_encoders = {}

for col in categorical_features:
    le = LabelEncoder()
    drug_df[col] = le.fit_transform(drug_df[col])
    label_encoders[col] = le

X = drug_df.drop(columns=["Drug"])
y = drug_df["Drug"]

y_le = LabelEncoder()
y = y_le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted')
rec = recall_score(y_test, y_pred, average='weighted')
```

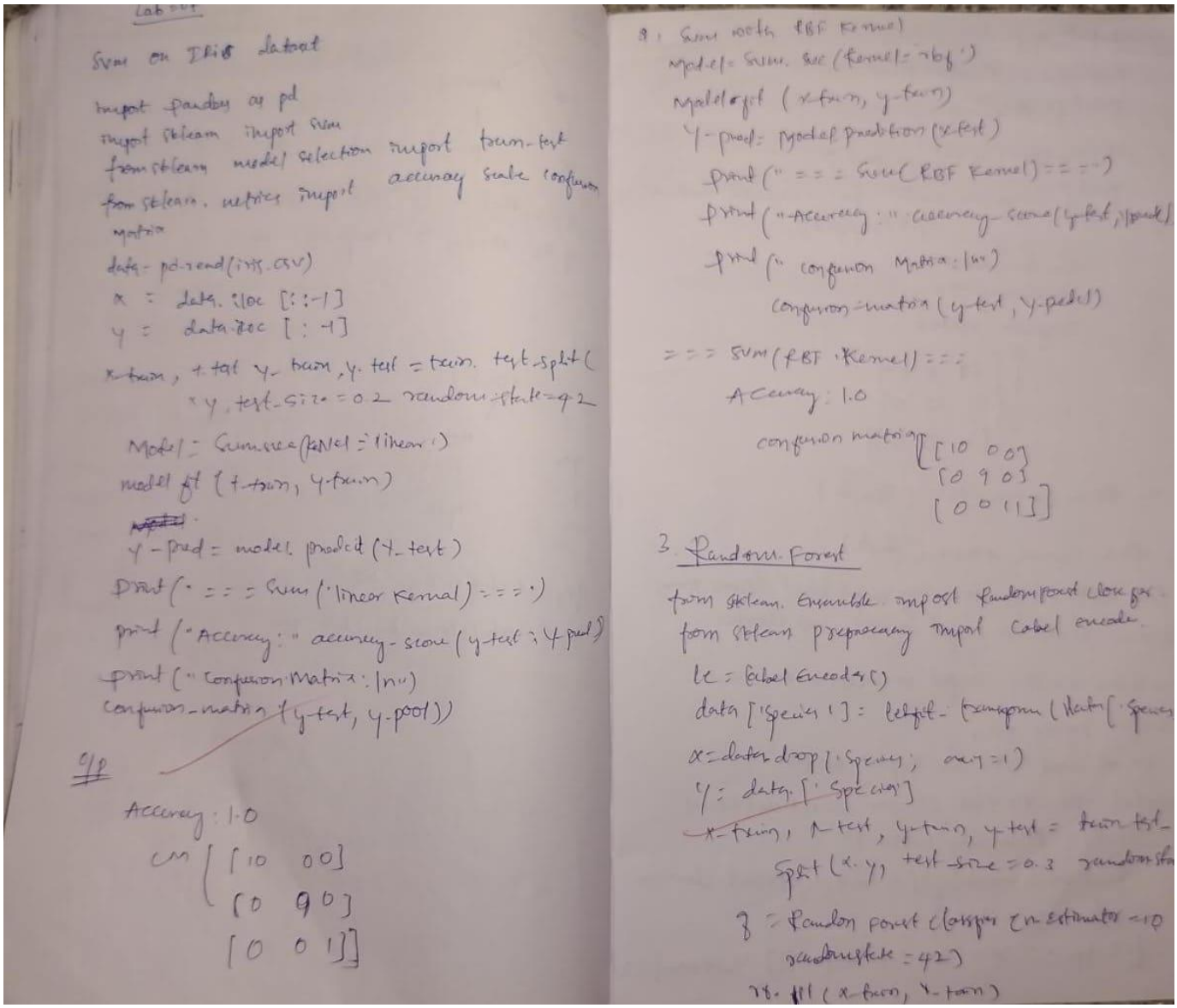
```
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)

    print("Drug Dataset Classification:")
    print(f"Accuracy Score: {acc:.4f}")
    print(f"Precision Score: {prec:.4f}")
    print(f"Recall Score: {rec:.4f}")
    print(f"F1 Score: {f1:.4f}")

  plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=y_le.classes_,
                yticklabels=y_le.classes_)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix: drug.csv")
    plt.show()

train_and_evaluate_drug()


petrol_df = pd.read_csv("petrol_consumption.csv")
X = petrol_df.drop(columns=["Petrol_Consumption"])
y = petrol_df["Petrol_Consumption"]
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled, y, test_size=0.2, random_state=42

)

model = DecisionTreeRegressor(max_depth=5, random_state=42)

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print("Petrol Consumption Regression:")

print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))

print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))

print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Program 7

Build Support vector machine model for a given dataset

Screenshot:



Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
```

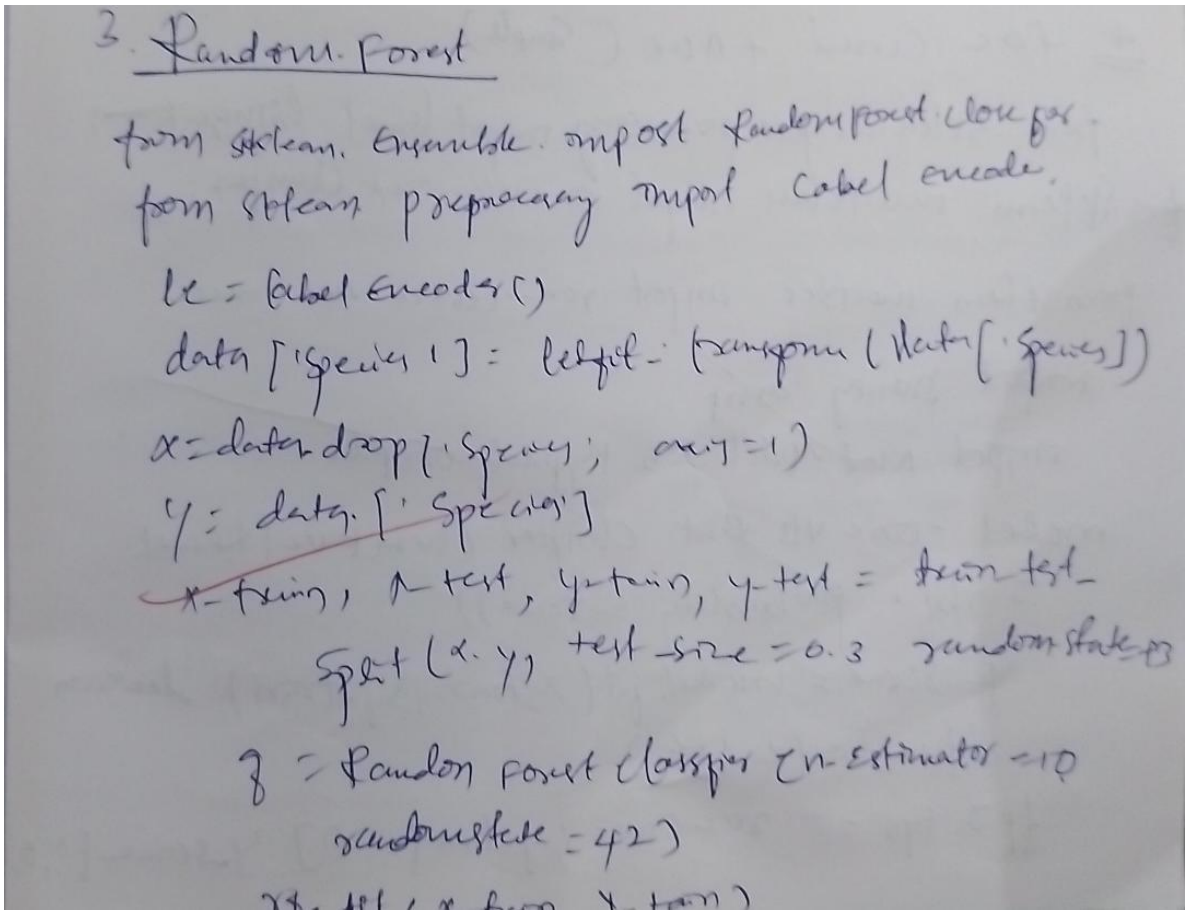
```
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc

from sklearn.preprocessing import label_binarize

import matplotlib.pyplot as plt

df = pd.read_csv("letter-recognition.csv")

df.columns = ["letter","xbox","ybox","width","height","onpix","xbar","ybar",

            "x2bar","y2bar","xybar","x2ybar","xy2bar","xedge","xedgey","yedge","yedgex"]


X = df.drop("letter", axis=1)

y = df["letter"]


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)

model = SVC(kernel='rbf', probability=True, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

classes = sorted(y.unique())

y_test_bin = label_binarize(y_test, classes=classes)

y_score = model.predict_proba(X_test)

fpr, tpr, _ = roc_curve(y_test_bin.ravel(), y_score.ravel())

roc_auc = auc(fpr, tpr)
```

```
plt.plot(fpr, tpr, linestyle=':', linewidth=3)

plt.plot([0,1], [0,1], 'k--')

plt.xlabel("FPR")

plt.ylabel("TPR")

plt.title("ROC Curve")

plt.show()

print("AUC Score:", roc_auc)
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot:



3. Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn preprocessing import LabelEncoder
le = LabelEncoder()
data['species'] = le.fit_transform(data['species'])
x = data.drop('species', axis=1)
y = data['species']
x_train, x_test, y_train, y_test = train_test_
    split(x, y, test_size=0.3 random_state=
z = RandomForestClassifier(n_estimators=10
    random_state=42)
```

```
y_pred = rf.predict(x_test)
print("=== Random Forest ===")
print("Accuracy:",
accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix,
                y_test, y_pred))
```

```
== Random Forest ==
Accuracy = 1.0
Confusion Matrix
[[ 10  0  0]
 [ 0  13  0]
 [ 0  0  13]]
```

4. ROC curve + AUC (Simple)

4 ROC Curve + AUC (Simple)

```
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier.
from sklearn.metrics import roc_curve, auc, curve.
import numpy as np
import matplotlib.pyplot as plt
model = OneVsRestClassifier(SVM.SVC(kernel
    = 'rbf', probability = True))
y_score = model.fit(x_train, y_train).decision
    function(x_test)
fpr, tpr, _ = roc_curve(y_bin[:, 0], y_score[:, 0])
```

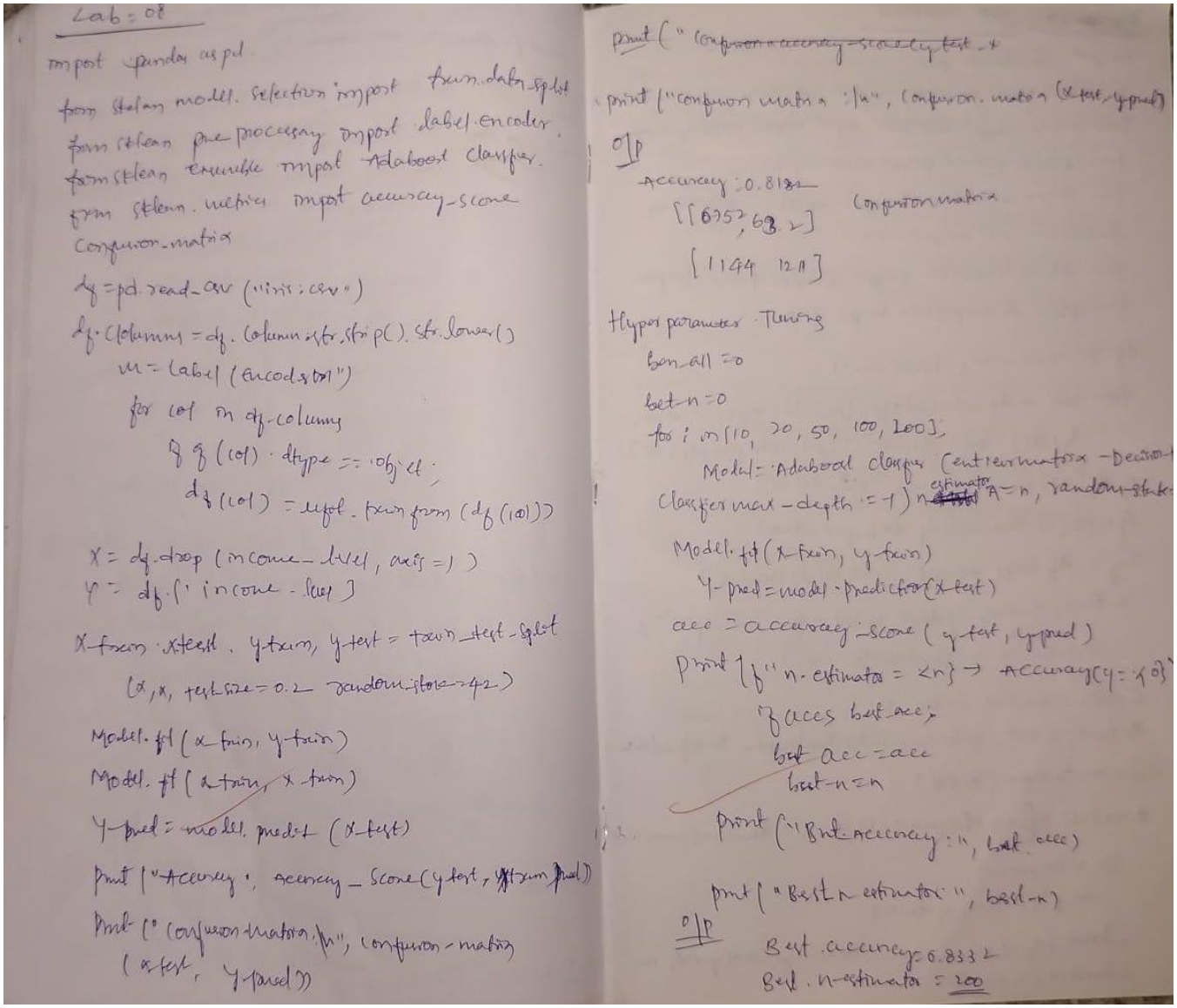
Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import confusion_matrix

iris_df = pd.read_csv("iris.csv")

X = iris_df.drop('species', axis=1)

y = iris_df['species']

le = LabelEncoder()

y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.3, random_state=42)

rf_model = RandomForestClassifier(n_estimators=10, random_state=42)

rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

print("Random Forest Accuracy with 10 trees:", accuracy_score(y_test, y_pred))

scores = []

n_range = range(1, 101)

best_model = None

best_preds = None

for n in n_range:model = RandomForestClassifier(n_estimators=n, random_state=42)
```

```
model.fit(X_train, y_train)

preds = model.predict(X_test)

acc = accuracy_score(y_test, preds)

scores.append(acc)

if acc == max(scores):

best_model = model

best_preds = preds

best_score = max(scores)

best_n = n_range[scores.index(best_score)]

print(f"Best Random Forest Accuracy: {best_score:.4f} with {best_n} trees")

plt.figure(figsize=(10, 5))

plt.plot(n_range, scores, marker='o', linestyle='-', color='blue')

plt.title('Random Forest Accuracy vs Number of Trees (Iris Dataset)')

plt.xlabel('Number of Trees')

plt.ylabel('Accuracy')

plt.grid(True)

plt.show()

cm = confusion_matrix(y_test, best_preds)

plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_, yticklabels=le.classes_)

plt.title(f"Confusion Matrix for Best Random Forest Model ({best_n} Trees)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

Program 9

Implement Boosting ensemble method on a given dataset.



Code:

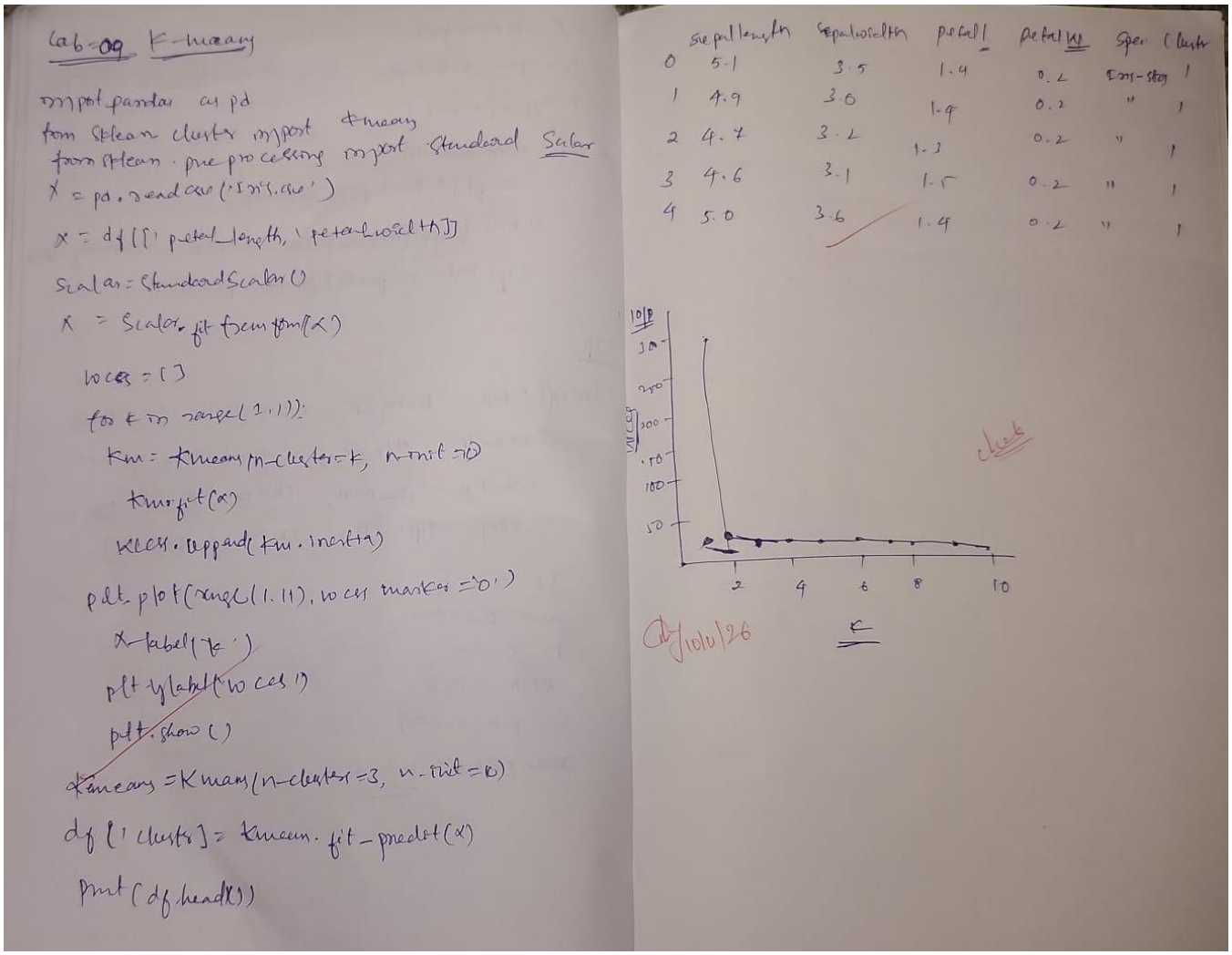
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.ensemble import AdaBoostClassifier
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix

income_df = pd.read_csv("income.csv")
X_income = income_df.drop('income_level', axis=1)
y_income = income_df['income_level']
X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(X_income, y_income, test_size=0.3,
random_state=42)

ada_model = AdaBoostClassifier(n_estimators=10, random_state=42)
ada_model.fit(X_train_i, y_train_i)
y_pred_i = ada_model.predict(X_test_i)
print("AdaBoost Accuracy with 10 estimators:", accuracy_score(y_test_i, y_pred_i))
scores_ada = []
n_range_ada = range(1, 51)
best_model_ada = None
best_preds_ada = None
for n in n_range_ada:
model = AdaBoostClassifier(n_estimators=n, random_state=42)
model.fit(X_train_i, y_train_i)
preds = model.predict(X_test_i)
acc = accuracy_score(y_test_i, preds)
```

```
scores_ada.append(acc)

if acc == max(scores_ada):

best_model_ada = model

best_preds_ada = preds


best_score_ada = max(scores_ada)

best_n_ada = n_range_ada[scores_ada.index(best_score_ada)]

print(f"Best AdaBoost Accuracy: {best_score_ada:.4f} with {best_n_ada} estimators")

plt.figure(figsize=(10, 5))

plt.plot(n_range_ada, scores_ada, marker='o', linestyle='-', color='orange')

plt.title('AdaBoost Accuracy vs Number of Estimators (Income Dataset)')

plt.xlabel('Number of Estimators')

plt.ylabel('Accuracy')

plt.grid(True)

plt.show()

cm_ada = confusion_matrix(y_test_i, best_preds_ada)

plt.figure(figsize=(6, 5))sns.heatmap(cm_ada, annot=True, fmt='d', cmap='Oranges', xticklabels=[0, 1],
yticklabels=[0, 1])

plt.title(f"Confusion Matrix for Best AdaBoost Model ({best_n_ada} Estimators)")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.



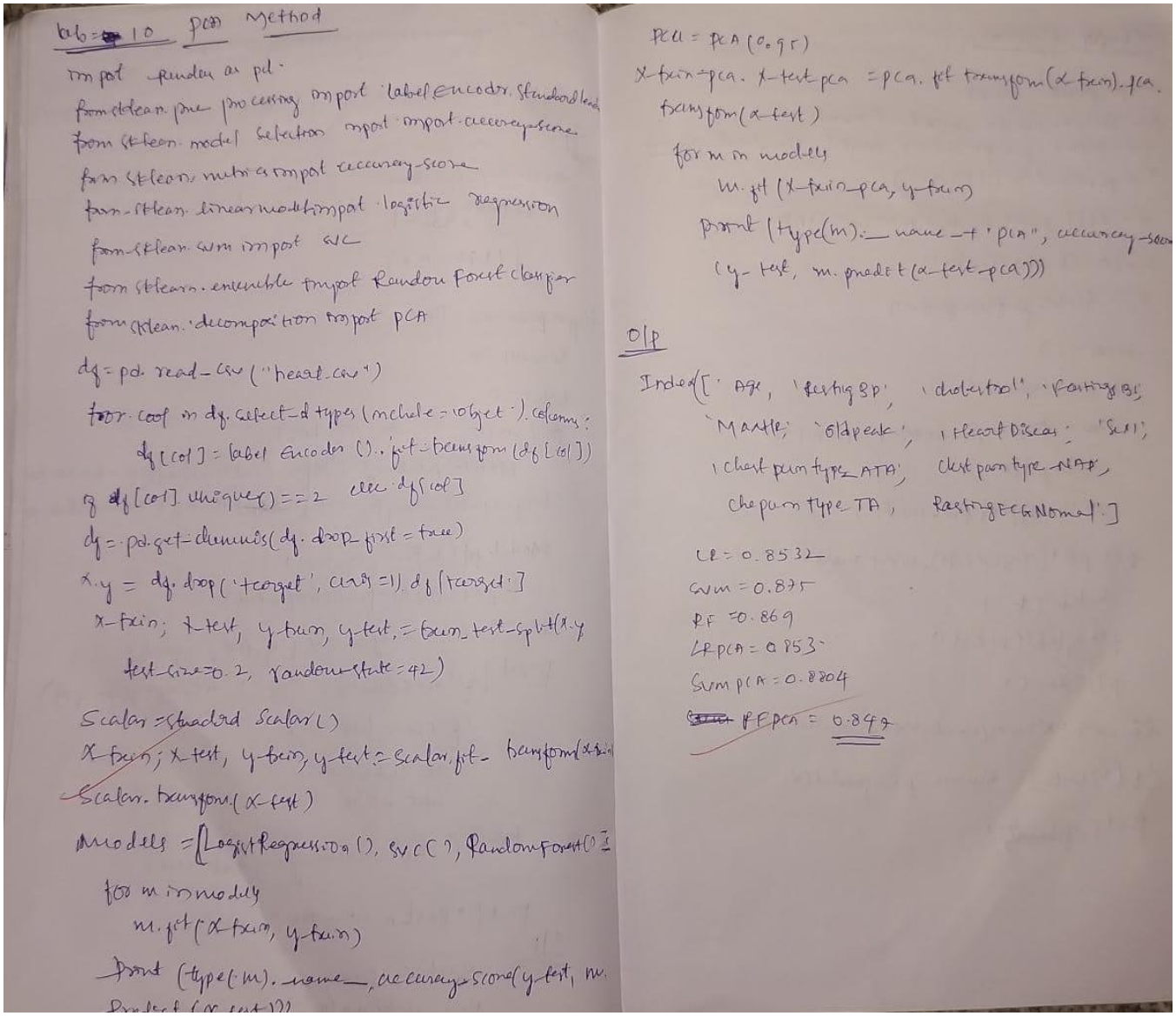
Code:

```
import pandas as pd

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt


df = pd.read_csv("/content/iris (1) (1).csv")
```

```
X = df[['petal_length', 'petal_width']]

scaler = StandardScaler()

X = scaler.fit_transform(X)

wcss = []

for k in range(1, 11):

    km = KMeans(n_clusters=k, n_init=10)

    km.fit(X)

    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, marker='o')

plt.xlabel('K')

plt.ylabel('WCSS')

plt.show()

kmeans = KMeans(n_clusters=3, n_init=10)

df['cluster'] = kmeans.fit_predict(X)

print(df.head())
```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.



lab = 10 PCA Method

import pandas as pd

from sklearn.preprocessing import label Encoder, StandardScaler

from sklearn.model selection import train-test-split

from sklearn.metrics import accuracy-score

from sklearn.linear model import logistic regression

from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier

from sklearn.decomposition import PCA

df = pd.read_csv("heart.csv")

for col in df.select_dtypes(include=['object']).columns:

df[col] = LabelEncoder().fit_transform(df[col])

if df[col].unique() == 2 else df[col]

df = pd.get_dummies(df, drop_first=True)

X, y = df.drop('target', axis=1), df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Scaler = StandardScaler()

X_train, X_test, y_train, y_test = Scaler.fit_transform(X_train), Scaler.transform(X_test)

models = [LogisticRegression(), SVC(), RandomForestClassifier()]

for m in models:

m.fit(X_train, y_train)

print(type(m).__name__, accuracy_score(y_test, m.predict(X_test)))

PCA = PCA(0.95)

X_train_pca, X_test_pca = pca.fit_transform(X_train), pca.transform(X_test)

for m in models:

m.fit(X_train_pca, y_train)

print(type(m).__name__ + "PCA", accuracy_score(y_test, m.predict(X_test_pca)))

O/P

Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease', 'Sex', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA', 'RestingECG_Normal'])

LR = 0.8532

SVM = 0.875

RF = 0.869

LRPCA = 0.853

SVMPCA = 0.8804

RFPCA = 0.847

Code:

```
import pandas as pd

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt
```

```
df = pd.read_csv("/content/iris (1) (1).csv")

X = df[['petal_length', 'petal_width']]

scaler = StandardScaler()

X = scaler.fit_transform(X)

wcss = []

for k in range(1, 11):

    km = KMeans(n_clusters=k, n_init=10)

    km.fit(X)

    wcss.append(km.inertia_)

plt.plot(range(1, 11), wcss, marker='o')

plt.xlabel('K')

plt.ylabel('WCSS')

plt.show()

kmeans = KMeans(n_clusters=3, n_init=10)

df['cluster'] = kmeans.fit_predict(X)

print(df.head())
```